

Initial Idea - GitHub Engineering Intelligence

The brief: A tool that connects to a team's GitHub repository and tells them **what's off track, why it happened, and what to do about it.**

The Flow, Explained by Example

Let's take a realistic GitHub project and walk through exactly what the application would do.

Example: E-commerce Backend Team

A company has the repository `acme-commerce/backend`. The team has 6 developers, currently building a new **Payment Service**.

Metric	Count
Open PRs...	18
Open issues...	42
Reviewers...	3
Milestone deadline...	5 days away

The engineering lead connects this repository to your application.

1 Connect GitHub

The user opens the application:

Engineering Intelligence

Connect your GitHub re

They authenticate with GitHub and select `acme-commerce/backend`. Your backend now has permission to read the required GitHub data.

2 Your Backend Collects GitHub Data

Your Python backend starts synchronizing.

GitHub

Pull Requests

Issues

Reviews

Commits

Milestones

Contributors

GitHub tells you, for example:

PR	Created	First review	Merged
#142	Aug 3	Aug 4	Aug 5
#143	Aug 4	Aug 6	Aug 7

#144	Aug 5	no review yet...	-
------	-------	------------------	---

You store this information in your database.

3 Analytics Engine Calculates Metrics

Now your Python analytics engine starts working. It calculates **PR cycle time**:

Previous 30 days:	Average = 18 hours	Current week:	Average = 34 hours
-------------------	--------------------	---------------	--------------------

$34 - 18 * 100 = +88\%$	18
-------------------------	----

PR cycle time increased by 88%.

4 It Doesn't Immediately Blame the PR Authors

This is where the system becomes interesting. It asks: *why* did PR cycle time increase?

So it breaks the metric down:

PR cycle time	Coding time	Waiting for review	Waiting for merg
---------------	-------------	--------------------	------------------

Suppose it discovers:

Phase	Before	After	Change
Coding time	12h	14h	+17%
Review waiting time	6h	20h	+233%...
Merge waiting time	0.5h	1h	+100%

The primary cause appears to be review delays.

5 Detection Engine Investigates Further

Now it examines reviewer activity:

Reviewer	Reviews
Rahul	15
Priya	3
Amit	2

And it finds:

- **7 PRs** are currently waiting for review
- **5 of those 7** are assigned to Rahul

The system creates a finding:

```
{  "type": "REVIEW_BOTTLENECK",  "severity": "HIGH",  "confidence": 0.91,  "evidence": [
```

Notice something important: the detection engine doesn't need an LLM. This is normal Python logic + SQL + statistics.

6 Health Score Changes

Suppose your health model was:

Dimension	Score
Delivery	85
Reviews	82
Issues	78
PR Quality	84
Activity	80
Overall	82 / 100 Healthy...

After the review bottleneck:

Dimension	Score	
Delivery	61	...
Reviews	42	...
Issues	76	...
PR Quality	83	...
Activity	79	...
Overall	67 / 100	At Risk...

7 Recommendation Engine

Your **deterministic** recommendation engine says:

Problem Review bottleneck

Evidence

- 7 PRs waiting
- Review time +233%
- 5/7 pending PRs assigned to Rahul

Possible actions

1. Redistribute pending reviews.
2. Prioritize PRs **#144** and **#147**.

3. Add another reviewer to the payment-service area.

At this point, you already have a useful product **without any AI**.

8 Now AI Comes In

The LLM receives the **structured analysis** - not *"go look at this GitHub repository and figure everything out."*

Instead, we give it something like:

```
{  "project_health": 67,    "previous_health": 82,    "findings": [          {                                "proble
```

The LLM's job is now purely to **explain** this. It might produce:

Project at risk: Review bottleneck

The biggest delivery problem this week is PR review time. Average review waiting time has increased by **233%**, and **7 PRs** are currently waiting for review.

Most of the pending review workload is concentrated on one reviewer, which is likely contributing to the delay.

Recommended next steps

- Redistribute the pending reviews.
- Prioritize the oldest payment-service PRs.
- Add another reviewer for payment-related changes.

Your application displays this on the dashboard.

9 The Tech Lead Asks a Question

Now we add the really cool part. The tech lead sees:

```
Project Health67 / 100  Review bottleneckReview time  233%[ Why? ]  [ What should I do? ]
```

They click **"Why?"** - or simply type *"Why is our project at risk?"*

Your system can answer using your **existing analysis**:

```
User  "Why is our project at risk?"  LLM  Need project health information  Backend / MCP tool  ge
```

This is where **MCP** becomes useful.

Where MCP Fits

Suppose we expose these tools:

Tool	Returns
get_project_health()	Overall + per-dimension scores
get_project_metrics()	Raw computed metrics
get_project_risks()	Active risks, ranked
get_open_prs()	Open PRs with age/state
get_stale_issues()	Issues past age threshold
get_review_bottlenecks()	Reviewer load + pending queue
get_recommendations()	Suggested actions

The AI can call them when answering questions. For example:

User: "What should we fix first?"

```
LLM I need current project risks. get_project_risks() Review bottleneck · Issue backlog · Larg
```

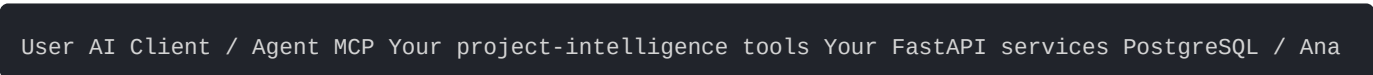
MCP isn't doing the analysis. It's giving the AI a standardized way to access your application's capabilities.

Complete Flow



One correction to that diagram

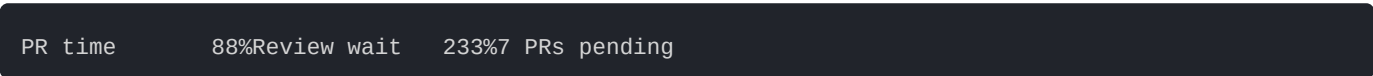
MCP would normally sit **between an MCP-compatible AI client/model and your application's tools** - rather than being something the LLM itself "contains." Conceptually:



And Here's the Important Distinction

We effectively have **three brains**:

1. Analytics Engine - "What happened?"



2. Detection + Recommendation Engine - "What's probably wrong and what should we do?"

Review bottleneck detected. Redistribute reviews. Prioritize oldest PRs.

3. AI / LLM - *"Explain it naturally."*

"The main reason delivery has slowed is..."

That's the Architecture to Aim For

It prevents the project from becoming:

"I sent GitHub data to ChatGPT."

Instead, you're building a **genuine engineering intelligence system with an AI interface**. That's a much stronger SDE-1 project.